

Reviewer Pack — Minimal Order of Quotient Space Manifolds and Circuit Monoto...

Entry 3dd4dfbd0d7b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 11:29:26 UTC

Minimal Order of Quotient Space Manifolds and Circuit Monotone Width Inequality

Entry ID: 3dd4dfbd0d7b Verdict: INCONCLUSIVE Recorded: 2026-06-06 11:29:26 UTC

1. Conjecture

Field A (mathematical branch): Geometric Topology (Quotient Space Theory) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every d -regular graph G , the minimal order of the quotient space manifold $M(G)$ constructed from G is linearly correlated with its circuit monotone width $w(M(G))$, such that $\text{ord}(M(G)) = \Theta(w(M(G)))$.

Rationale (proposer’s reasoning):

Quotient spaces in geometric topology have been used to simplify complex structures. If the structure of the quotient space manifold can be related to the complexity of computing properties of the original graph, it may reveal new insights into circuit complexity.

Taxonomy category: quotient_space_manifold (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3e8b3fdbf3520eaa

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each graph G , if Pearson’s correlation coefficient of $\text{ord}(M(G))$ and $w(M(G))$ across 30 seeds is greater than 0.5, support the conjecture; otherwise, falsify it.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal order quotient space manifold" AND "circuit monotone width" - "Quotient Space Theory" AND "Boolean Circuit Complexity" - "geometric topology" AND "circuit monotone width inequality"

Top relevant hits considered: - [s2:10.1007/978-0-387-78901-9_12] Fixed-point theorems

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

# Helper functions for graph operations
def generate_d_regular_graph(n, d):
    if n * d % 2 != 0:
        raise ValueError("Graph size must be a multiple of the degree")

    graph = [[] for _ in range(n)]
    edges_added = set()

    for node in range(n):
        for neighbor in range(node + 1, n):
            if len(graph[node]) >= d or len(graph[neighbor]) >= d:
                continue
            if (node, neighbor) not in edges_added and (neighbor, node) not in edges_added:
                graph[node].append(neighbor)
                graph[neighbor].append(node)
                edges_added.add((node, neighbor))

    return graph

def dfs(graph, start, parent):
    stack = [start]
    visited = set()
    max_depth = 0

    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            for neighbor in graph[node]:
                if neighbor != parent:
```

```

        max_depth = max(max_depth, dfs(graph, neighbor, node))

    return max_depth + 1

def circuit_monotone_width(graph):
    n = len(graph)
    width = 0

    for i in range(n):
        width = max(width, dfs(graph, i, -1))

    return width

# Main function to run a trial
def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []

    for n in [5, 10, 15, 20, 30, 40]:
        graph = generate_d_regular_graph(n, 3)
        width = circuit_monotone_width(graph)
        order = len(graph) # Simplified order as the number of vertices

        results.append({
            "n": n,
            "width": width,
            "order": order
        })

    if not results:
        return {
            "metric_name": "circuit_monotone_width",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "no_graphs_generated"
        }

    total_order = sum(result["order"] for result in results)
    total_width = sum(result["width"] for result in results)
    instances_tested = len(results)
    n_max = max(result["n"] for result in results)

    if instances_tested < 30:
        return {
            "metric_name": "circuit_monotone_width",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "insufficient_instances"
        }

    mean_order = total_order / instances_tested
    mean_width = total_width / instances_tested

```

```

correlation_coefficient = (instances_tested * sum(order * width for result in results) -
                           mean_order * total_width - mean_width * total_order) / \
                           math.sqrt((instances_tested * sum(order**2 for result in results) -
                                   (instances_tested * sum(width**2 for result in results) -

return {
    "metric_name": "circuit_monotone_width",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": abs(correlation_coefficient) > 0.5,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79]

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        if "conjecture_holds" in trial_result and not trial_result["conjecture_holds"]:
            break

    if all("conjecture_holds" in result and result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
        support_fraction = sum(1 for result in results if "conjecture_holds" in result and result["conjecture_holds"]) / len(results)

        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any("counterexample" in result and result["counterexample"] for result in results):
        counterexample = next(result["counterexample"] for result in results if "counterexample" in result and result["counterexample"])
        first_failing_seed = next(result["seed"] for result in results if "conjecture_holds" in result and not result["conjecture_holds"])

        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c025bb75.py", line 127, in <module>
    trial_result = run_trial(seed)
                  ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c025bb75.py", line 67, in run_trial

```


4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3dd4dfbd0d7b.tar.gz` (if generated)